SolarShare — Every Concept Explained Simply

> **Purpose:** Understand what we are actually building, how each piece works, and how they connect.
> Written in plain language so every team member can explain any part confidently.

Part 1: The Big Picture — What Are We Building?

The One-Line Answer

We are building an **app** (like Swiggy, but for electricity) that lets your solar-powered neighbor **sell you their extra electricity** at a price cheaper than the government electricity company, and lets them **earn more** than they currently do.

The Real-World Analogy

Imagine you grow mangoes on your terrace. Right now, your only option is to sell extras to the government mandi at Rs.20/kg. Your neighbor wants mangoes and buys from the market at Rs.50/kg. You both lose.

SolarShare is like a local marketplace: your neighbor buys directly from you at Rs.35/kg. You earn Rs.15 more. They save Rs.15. Nobody needs the mandi in the middle.

Now replace "mangoes" with "electricity" and "mandi" with "DISCOM" — that's exactly what SolarShare does.

What We DON'T Build

This is important to understand:

We DON'T build	Who builds it	Status
Solar panels	PM Surya Ghar scheme	50 lakh+ households already have them
Smart meters	DISCOM (under RDSS scheme)	7.24 crore already installed
The electricity grid	DISCOM	Already exists everywhere
The trading protocol	Government (India Energy Stack)	Already live, demoed to PM
Payment infrastructure	UPI/NPCI	Already exists

We ONLY build the app that connects buyers and sellers. Everything else already exists.

Part 2: Key Players — Who's Who?

The Three Users

1. Prosumer (Producer + Consumer)

- A household that has rooftop solar panels (installed under PM Surya Ghar)
- On sunny days, they produce MORE electricity than they use
- This "extra" electricity is called **surplus**
- Currently, they can only sell surplus to the DISCOM at a low price
- SolarShare lets them sell to their neighbor at a better price

2. Buyer (Consumer without solar)

- A household in the same society that does NOT have solar panels
- They pay the full DISCOM retail tariff (e.g., Rs.6/kWh)
- SolarShare lets them buy from a solar neighbor at a cheaper rate (e.g., Rs.5/kWh)
- They save money AND use green energy

3. RWA Admin (Coordinator)

- The Resident Welfare Association coordinator of the housing society
- They are the trusted person who says "this is legit, everyone should join"
- They help with bulk onboarding — getting 50-200 households on the app at once
- They don't trade energy themselves, they facilitate adoption

The Institutions

DISCOM (Distribution Company)

- The government/private company that runs the electricity grid in your area
- Examples: BSES Rajdhani (Delhi), Tata Power Delhi, PVVNL (Western UP)
- They own the wires, meters, and billing systems
- In P2P trading, they still maintain the grid — they just lose their middleman role for solar surplus

DERC / UPERC (Regulators)

- Delhi Electricity Regulatory Commission / UP Electricity Regulatory Commission
- They are the government bodies that APPROVE whether P2P trading is legal
- In February 2026, they approved a 6-month pilot allowing P2P trading
- Without their approval, we cannot legally operate

REC Limited (Nodal Agency)

- The government agency that manages the India Energy Stack
- To operate as an app on the energy stack, we need REC's authorization (BAP registration)

Part 3: The India Energy Stack — Our Foundation

What Is It?

The India Energy Stack (IES) is a **government-built digital system** that makes energy trading possible, just like:

System	What It Did	For What
UPI	Created an open standard for money transfer	Payments (PhonePe, GPay, Paytm all use UPI)
ONDC	Created an open standard for buying/selling goods	E-commerce (any app can sell to any buyer)
IES	Created an open standard for energy trading	Electricity (any app can trade energy)

The key insight: **UPI didn't build the banks. ONDC didn't build the shops. IES doesn't build the power grid.** They just built the common language that lets everyone talk to each other.

The Layers

Think of IES like a 5-layer cake:

```
Layer 5: USERS (that's us! and the prosumers/buyers)
         We build the app people actually see and use

Layer 4: APPLICATION LAYER (SolarShare sits here)
         Apps that connect to the energy network
         We register as a "BAP" (Beckn Application Provider)

Layer 3: NETWORK LAYER (Unified Energy Interface / UEI)
         The "UPI of energy" - common protocol
         Uses Beckn Protocol (same family as ONDC)
         Govt-built, open-source, free to use

Layer 2: DATA LAYER (DISCOM systems)
         Smart meter data, billing systems
         AMISP manages the data pipeline
         DLMS/COSEM protocol for meter communication

Layer 1: PHYSICAL LAYER (grid, meters, solar panels)
         Smart meters on every home
         Wires connecting the society
         Solar panels on rooftops
```

We only build Layer 4. Everything else is already built by the government or DISCOMs.

Part 4: The Beckn Protocol — The Language Our App Speaks

What Is It?

The Beckn Protocol is an **open-source communication standard** — think of it as a language that all energy apps agree to speak, so they can understand each other.

It was invented in India. The same protocol family powers:

- **ONDC** (e-commerce)
- **UHI** (health)
- **DSEP** (education/skilling)
- **UEI** (energy) — this is what we use

The Two Sides

Term	Full Form	Role	Example
BAP	Beckn Application Provider	The "buyer-side" app	SolarShare
BPP	Beckn Provider Platform	The "seller/supplier-side" system	DISCOM's meter + billing systems

How a Trade Actually Happens (API Flow)

When a buyer opens SolarShare and wants to buy energy, here's what happens behind the scenes:

```
Step 1: SEARCH
  SolarShare (BAP) asks the network: "Any surplus solar available in Green Valley Society?"
  The Beckn Gateway broadcasts this to all relevant BPPs (DISCOMs)

Step 2: ON_SEARCH (callback)
  DISCOM's system responds: "Yes! Flat A-302 has 8.2 kWh available at Rs.5.10/kWh"

Step 3: SELECT
  Buyer picks this listing: "I want to buy from Flat A-302"

Step 4: ON_SELECT (callback)
  System confirms: "Here are the full details: price, quantity, transaction fee, etc."

Step 5: INIT
  SolarShare sends: "Initialize this trade. Here are the buyer's details."

Step 6: ON_INIT (callback)
  System responds: "Trade draft ready. Payment of Rs.41.82 required in escrow."

Step 7: CONFIRM
  Buyer confirms and payment is placed in escrow.

Step 8: ON_CONFIRM (callback)
  Trade is finalized, logged, hashed, and both parties are notified.
```

Important: This is **asynchronous**. Each step sends a request, gets an immediate "received!" (ACK), and then the actual answer comes back later via a callback. This is like ordering on Swiggy — you place the order, get a confirmation, and then the restaurant prepares it separately.

For Our Hackathon MVP

We **simulate** the Beckn flow. Instead of actually calling the UEI network, we:

1. Create our own backend that mimics the BPP responses
2. Build the BAP logic (search, select, init, confirm) so it's architecturally ready
3. When we register as a real BAP in Phase 2, we just point it to the real UEI instead of our mock

---

Part 5: Smart Meters — Where the Data Comes From

What Is a Smart Meter?

A smart meter is a **digital electricity meter** that:

- Measures how much electricity you **import** (take from the grid) and **export** (send back to the grid)
- Records this data every **15 minutes**
- Sends the data automatically to the DISCOM (no meter reader visits needed)
- Can be controlled remotely (connect/disconnect)

How Is It Different from a Normal Meter?

Feature	Normal Meter	Smart Meter
Reading	Manual (meter reader visits)	Automatic (sends data wirelessly)
Frequency	Once a month	Every 15 minutes
Direction	One-way (import only)	**Bidirectional** (import AND export)
Communication	None	Wireless (4G, RF, PLC)
Data	Just total kWh	Detailed load profile, voltage, current

The Data Pipeline

Here's how data flows from the meter to our app:

```
Smart Meter (on your wall)
    |
    | sends data via DLMS/COSEM protocol
    | (wirelessly: 4G / RF Mesh / Power Line Communication)
    |
Data Concentrator Unit (DCU) [optional, for RF Mesh]
    |
    | aggregates data from multiple meters
    |
Head-End System (HES)
    |
    | the "brain" that talks to all meters
    | collects data, sends commands
    | managed by AMISP
    |
Meter Data Management (MDM)
    |
    | validates and cleans the data
    | checks for errors, fills gaps
    | makes data "billing-ready"
    |
UEI / Beckn API Layer
    |
    | exposes the verified data as APIs
    | any registered BAP can access it
    |
SolarShare App
    |
    | displays surplus, matches trades
    |
Users see it on their phone
```

What Is DLMS/COSEM?

DLMS/COSEM is the **communication language** that smart meters speak. Just like all websites use HTTP and all emails use SMTP, all smart meters use DLMS/COSEM.

- **DLMS** = Device Language Message Specification (the grammar)
- **COSEM** = Companion Specification for Energy Metering (the vocabulary)
- Every data point in the meter has a unique **OBIS code** (like a serial number for each type of reading)
- India mandates this through **IS 15959** standard

You don't need to know the details of DLMS/COSEM for the hackathon. Just know that it's the standard protocol that ensures all meter data is structured the same way, regardless of which company made the meter.

What Is AMISP?

AMISP = Advanced Metering Infrastructure Service Provider

This is the company that the DISCOM hires to:

1. Install smart meters
2. Set up the communication network
3. Run the Head-End System (HES)
4. Maintain everything and ensure data keeps flowing

They get paid per meter per month and have strict SLAs (Service Level Agreements) for data accuracy and uptime.

Key insight for SolarShare: We never talk to the meter directly. We talk to the AMISP's system (via the UEI layer), which gives us clean, verified data. This means we don't need to worry about meter hardware, communication protocols, or data quality — the AMISP handles all of that.

Part 6: The Trust Layer — SHA-256 and Blockchain Explained

This is the part most people find confusing. Let's break it down completely.

Why Do We Need a Trust Layer?

When Priya sells 8 kWh of solar energy to Rahul at Rs.5.10/kWh, both of them need to trust that:

1. The trade was recorded accurately (8 kWh, not 6 kWh)
2. Nobody can change it later (not Priya, not Rahul, not even SolarShare)
3. The DISCOM can verify it when adjusting their bills
4. If there's a dispute, there's an unquestionable record

Without trust = nobody will use the platform.

Level 1: SHA-256 Hashing (What We Use in the Hackathon)

What Is Hashing?

Hashing is like creating a **unique fingerprint** for any piece of data. No matter how much data you put in, you always get a fixed-length output.

```
Input:  "Priya sold 8.2 kWh to Rahul at Rs.5.10"
Output: "a7f2c9e81b3d4f6a8c2e1d0b9f7e5a3c..."  (64 characters, always)

Input:  "Priya sold 8.3 kWh to Rahul at Rs.5.10"  (changed 8.2 to 8.3)
Output: "f1b8d3e7c4a2906b5e8d1f3a7c9b2e4d..."  (COMPLETELY different!)
```

Key properties:

1. **One-way:** You CANNOT reverse the hash to get the original data
2. **Deterministic:** Same input always gives the same hash
3. **Avalanche effect:** Change even one character, and the entire hash changes
4. **Collision-resistant:** It's practically impossible to find two different inputs that give the same hash

What Is SHA-256 Specifically?

SHA-256 is a specific hashing algorithm (SHA = Secure Hash Algorithm, 256 = output is 256 bits = 64 hex characters). It's used by:

- Bitcoin (to mine blocks)
- SSL certificates (to secure websites)
- Digital signatures (to verify documents)
- SolarShare (to make trade records tamper-proof)

How We Build a Hash Chain

Instead of hashing each trade independently, we **chain** them together:

```
TRADE #1 (Genesis — the first ever trade)
+-------------------------------------------+
| Data: Priya -> Rahul, 8.2 kWh, Rs.5.10    |
| Previous Hash: "0000000000" (no previous) |
| THIS Hash: "a7f2c9..."                    |
+-------------------------------------------+
        |
        | (this hash goes into the next trade)
        v
TRADE #2
+-------------------------------------------+
| Data: Sanjay -> Meera, 5.0 kWh, Rs.4.90   |
| Previous Hash: "a7f2c9..." (from Trade 1) |
| THIS Hash: "3e8b1f..."                    |
+-------------------------------------------+
        |
        | (this hash goes into the next trade)
        v
TRADE #3
+-------------------------------------------+
| Data: Priya -> Vikram, 3.1 kWh, Rs.5.20   |
| Previous Hash: "3e8b1f..." (from Trade 2) |
| THIS Hash: "c4d7a2..."                    |
+-------------------------------------------+
```

Why Is This Tamper-Proof?

Scenario: Someone tries to change Trade #1 from 8.2 kWh to 15 kWh.

```
BEFORE tampering:
  Trade 1 hash: "a7f2c9..."  <-- Trade 2 stores this
  Trade 2 hash: "3e8b1f..."  <-- Trade 3 stores this

AFTER tampering Trade 1:
  Trade 1 hash: "x9y8z7..."  <-- CHANGED (because data changed)
  But Trade 2 still has: Previous = "a7f2c9..."  <-- MISMATCH!

  The chain is BROKEN. Everyone can see the tampering.
```

To successfully tamper without detection, you'd need to recalculate the hash of Trade #1, then Trade #2, then Trade #3, then every trade after that. And you'd need to do this on every copy of the ledger simultaneously. This is computationally infeasible.

The Actual Code (Node.js)

```javascript
const crypto = require('crypto');

// This function creates a tamper-evident hash for each trade
function createTradeRecord(tradeData, previousHash) {
  // Combine the trade data with the previous hash
  const combined = JSON.stringify(tradeData) + previousHash;

  // Create SHA-256 hash
  const hash = crypto.createHash('sha256').update(combined).digest('hex');

  return {
    ...tradeData,
    previousHash: previousHash,
    hash: hash,
    timestamp: new Date().toISOString()
  };
}

// Example usage:
const genesis = createTradeRecord(
  { seller: "A-302", buyer: "D-201", kWh: 8.2, price: 5.10 },
  "0000000000000000"  // Genesis block has no previous
);

const trade2 = createTradeRecord(
  { seller: "B-105", buyer: "C-401", kWh: 5.0, price: 4.90 },
  genesis.hash  // Chain to previous trade
);

// To verify: recalculate the hash and compare
function verifyChain(trades) {
  for (let i = 1; i < trades.length; i++) {
    const expected = trades[i].previousHash;
    const actual = trades[i - 1].hash;
    if (expected !== actual) {
      return { valid: false, brokenAt: i };
    }
  }
  return { valid: true };
}
```

Level 2: Blockchain (What We Use in Production)

Hash Chain vs. Blockchain — What's the Difference?

Feature	Hash Chain (MVP)	Blockchain (Production)
Who stores the data?	Our server only	Multiple organizations' servers
Who can verify?	Only us	Everyone on the network
Can WE tamper?	Technically yes (we control the only copy)	No (we'd need to hack every copy)
Complexity	Simple — 10 lines of code	Complex — needs Hyperledger Fabric setup
Cost	Free	Requires infrastructure
Good enough for	Hackathon demo	Real money, real trades

Think of it this way:

- A hash chain is like a diary with a lock — YOU control the diary
- A blockchain is like a shared Google Doc with everyone having a copy — nobody alone can change it

Why Hyperledger Fabric (Not Bitcoin/Ethereum)?

Feature	Bitcoin/Ethereum (Public)	Hyperledger Fabric (Permissioned)
Who can join?	Anyone	Only authorized participants
Privacy	Everything is public	Data can be private between parties
Speed	7-15 transactions/second	3,000+ transactions/second
Cost per transaction	Gas fees (expensive)	No gas fees (free after setup)
Energy consumption	Very high (mining)	Very low (no mining)
Identity	Anonymous	Known, verified identities

For energy trading, we need:

- Privacy (DPDP Act compliance) -> Hyperledger has private data collections
- Speed (many trades during peak solar hours) -> Hyperledger is fast
- No cost per trade (small trades, low margins) -> No gas fees
- Known identities (verified DISCOM customers) -> Permissioned network

How Hyperledger Fabric Works — The Analogy

Think of it like a **shared Excel spreadsheet** with strict rules:

1. **Organizations** = Different companies/entities that each run their own copy of the spreadsheet
    - SolarShare (runs the app)
    - DISCOM (verifies meter data)
    - RWA (community governance)
    - Regulator (audit access)

2. **Channel** = A specific spreadsheet that only certain organizations can see
    - "green-valley-energy" channel: only SolarShare, BRPL, and Green Valley RWA
3. **Chaincode (Smart Contract)** = The rules/formulas in the spreadsheet that run automatically

```
RULE: When a trade is submitted...
  1. Check: Does the seller have enough surplus? (verify with DISCOM's meter data)
  2. Check: Has the buyer's payment been placed in escrow?
  3. If both yes: Record the trade, notify both parties
  4. If either no: Reject the trade, return escrow
```

4. **Endorsement Policy** = How many organizations must "approve" a trade before it's recorded

```
Policy: "A trade is valid only if BOTH SolarShare AND the DISCOM endorse it"
```

   This means even if SolarShare tries to create a fake trade, the DISCOM won't endorse it, so it fails.

5. **Ledger** = The actual spreadsheet data
    - Every organization has a copy
    - All copies are synchronized
    - Nobody can change their copy without everyone else's copy showing the change

How We'll Actually Implement It (Phase 2)

```
Step 1: Set Up the Network
  - Create a Fabric network with 2-3 organizations (SolarShare + DISCOM + RWA)
  - Each org runs a "peer" node (a server that stores a copy of the ledger)
  - Set up an "orderer" node (ensures all transactions are in the same order)

Step 2: Write the Chaincode (Smart Contract)
  - Written in Node.js (since our team knows JavaScript)
  - Functions:
    * listSurplus(sellerId, kWh, pricePerKWh) - prosumer lists energy
    * createTrade(buyerId, listingId, kWh) - buyer initiates trade
    * verifyMeterData(tradeId, meterReading) - DISCOM verifies
    * settleTrade(tradeId, paymentRef) - finalize and record
    * getTradeHistory(userId) - view past trades

Step 3: Deploy Chaincode to the Channel
  - Install the chaincode on each peer
  - Approve it from each organization
  - Commit it to the channel

Step 4: Connect the App
  - Use Hyperledger Fabric SDK (Node.js) to connect our backend to the blockchain
  - When a user trades, our app submits a transaction to the blockchain
  - The blockchain endorses, orders, and commits it
  - All copies update simultaneously

Step 5: Private Data Collections
  - Exact trade prices: visible only to buyer, seller, and DISCOM
  - Aggregate statistics: visible to everyone (for transparency)
  - User identities: verified but not publicly exposed
```

Tools We'd Use

Tool	Purpose
Hyperledger Fabric v2.x	The blockchain platform itself
Fabric SDK (Node.js)	To connect our app to the blockchain
VS Code + IBM Blockchain Extension	To develop and test chaincode locally
Hyperledger Caliper	To benchmark performance (how many trades/second?)
Docker	To run blockchain nodes in containers

Part 7: Payment Settlement — How Money Flows

The Problem

When Priya sells energy to Rahul, how does Rahul pay? They're neighbors, but:

- Direct cash payment between neighbors is messy and untrustworthy
- What if Rahul doesn't pay?
- What if Priya doesn't deliver?

The Solution: Escrow

Escrow means a trusted third party holds the money until both sides fulfill their obligations.

```
Step 1: Rahul wants to buy 8 kWh at Rs.5.10/kWh = Rs.41.82
Step 2: Rahul's payment (Rs.41.82) goes into ESCROW (held by payment gateway)
Step 3: The trade executes (smart meter confirms energy was exported)
Step 4: DISCOM verifies: "Yes, Priya did export 8 kWh"
Step 5: Escrow releases Rs.41.82 to Priya (minus transaction fee of Rs.0.21)
Step 6: Both parties' DISCOM bills are adjusted

If anything fails at Step 3 or 4: Escrow returns Rs.41.82 to Rahul
```

For the Hackathon MVP

We use a **mock wallet** — simulated balance for each user. Real escrow integration happens in Phase 2 with an RBI-compliant payment gateway.

How DISCOM Bills Get Adjusted

P2P trades are NOT separate cash transactions. Per the IES design, they are reflected as **cumulative adjustments** in both parties' regular DISCOM bills:

- **Priya's bill:** Your export of 8 kWh was traded via SolarShare. Credit: Rs.41.61 (after transaction fee)
- **Rahul's bill:** You purchased 8 kWh via SolarShare. Adjusted billing: -8 kWh from grid consumption.

This means the DISCOM still manages the billing relationship. SolarShare facilitates the trade, the DISCOM records the adjustment.

Part 8: Net Metering vs. P2P Trading — The Key Difference

Net Metering (What Exists Today)

```
Priya's Solar Panel generates 20 kWh
Priya uses 12 kWh herself
Surplus = 8 kWh

That 8 kWh goes to the grid
DISCOM credits Priya at Feed-in Tariff (e.g., Rs.4.10/kWh)
Priya earns: 8 x Rs.4.10 = Rs.32.80

Meanwhile, Rahul (next door) buys 8 kWh from DISCOM at retail (Rs.6/kWh)
Rahul pays: 8 x Rs.6 = Rs.48.00

DISCOM profit on this: Rs.48 - Rs.32.80 = Rs.15.20
```

P2P Trading via SolarShare (What We Enable)

```
Priya's Solar Panel generates 20 kWh
Priya uses 12 kWh herself
Surplus = 8 kWh

Priya lists 8 kWh on SolarShare at Rs.5.10/kWh
Rahul buys it:  8 x Rs.5.10 = Rs.40.80
Transaction fee: 8 x Rs.0.42 = Rs.3.36 (Rs.1.68 each)

Priya earns: Rs.40.80 - Rs.1.68 = Rs.39.12  (was Rs.32.80 -> +19% more)
Rahul pays:  Rs.40.80 + Rs.1.68 = Rs.42.48  (was Rs.48.00 -> 11.5% less)

Both win. The DISCOM still earns the transaction fee (Rs.3.36).
```

The Physical Reality

Important: The actual electrons don't travel directly from Priya's panel to Rahul's home. They can't — electricity follows the laws of physics, not contracts.

What actually happens:

1. Priya's surplus goes into the local grid (same as net metering)
2. Rahul draws from the local grid (same as always)
3. The **trade is an accounting transaction** — the billing is adjusted so that Priya gets credited at the agreed P2P rate instead of the feed-in tariff, and Rahul gets charged at the P2P rate instead of the retail tariff

The grid is the "battery." Surplus goes in, deficit comes out. The accounting layer (SolarShare + DISCOM billing) keeps track of who owes whom.

Part 9: Putting It All Together — The Complete Flow

Here's the entire journey, from sunlight to settlement:

```
MORNING: Sun rises

1. Priya's solar panels generate electricity
2. Her smart meter records: Import = 1.5 kWh, Export = 8.2 kWh
3. Meter sends data via DLMS/COSEM to DISCOM's Head-End System
4. MDM validates the data

5. Priya opens SolarShare app
6. App shows: "You have 8.2 kWh surplus available"
7. Priya lists it at Rs.5.10/kWh

8. SolarShare (as BAP) sends /search to UEI Beckn Gateway
9. Gateway responds with available listings

10. Rahul opens SolarShare app
11. Sees: "Flat A-302 has 8.2 kWh at Rs.5.10 — Save Rs.7.36 vs DISCOM!"
12. Rahul taps "Buy"

13. SolarShare sends /select, /init, /confirm through Beckn flow
14. Rahul's Rs.42.48 (including fees) goes into escrow

15. Trade is recorded:
    - Data: {seller: A-302, buyer: D-201, 8.2 kWh, Rs.5.10, timestamp}
    - Combined with previous trade's hash
    - SHA-256 hash generated: "a7f2c9e81b3d..."
    - Logged to database (MVP) / blockchain (production)

16. Both get notification: "Trade complete!"

17. DISCOM verifies against their meter data
18. Escrow releases payment to Priya (minus transaction fee)

19. At month end:
    - Priya's DISCOM bill shows: "P2P credit: Rs.39.12"
    - Rahul's DISCOM bill shows: "P2P purchase: 8.2 kWh adjusted"

EVENING: Sun sets, no more surplus
20. Rahul automatically falls back to standard grid supply
21. No interruption, no manual action needed
```

Part 10: What We Build in Each Phase

Phase 0 — Hackathon (1 Week) — THIS IS WHAT WE DEMO

What We Build	How
3 app screens (list, buy, ledger)	React Native / Flutter
Backend API	Node.js / FastAPI
Database for users and trades	Firebase / Supabase (free)
SHA-256 hash chain	Built-in crypto library, ~10 lines of code
Simulated meter data	JSON files mimicking 15-min interval readings
Mock payment wallet	Simple balance tracking, no real money
Price suggestion algorithm	Basic calculation: midpoint between feed-in and retail tariff

What we DON'T need:

- Real meters, real DISCOM data, real payments, real blockchain, real Beckn registration

Demo script:

1. Show Priya listing 8.2 kWh surplus at Rs.5.10
2. Show Rahul browsing and buying
3. Show the trade ledger with SHA-256 hashes
4. Show that changing any past trade breaks the hash chain (tamper detection)
5. Show the dashboard with trade history and savings calculated

Phase 1 — Pilot (Month 1-3)

- Partner with one real housing society in Delhi (DERC pilot zone)
- Replace simulated data with manual/CSV meter readings
- Onboard 50-200 households via RWA
- WhatsApp-based notifications

Phase 2 — Integration (Month 3-6)

- Register as BAP on UEI network
- Connect to real AMISP/HES data feed
- Implement Hyperledger Fabric blockchain
- Integrate RBI-compliant escrow payment

Phase 3 — Scale (Month 6-12)

- Expand to 5+ societies
- Add AI-based demand/surplus forecasting
- Launch carbon credit tracking
- Start multi-state expansion

Quick Reference Card

Term	What It Is	Simple Analogy
P2P Trading	Selling directly to your neighbor	Selling mangoes to your neighbor instead of the mandi
Prosumer	Someone who both produces and consumes energy	A household with solar panels
BAP	Beckn Application Provider	SolarShare's registration on the energy network
BPP	Beckn Provider Platform	DISCOM's system on the energy network
UEI	Unified Energy Interface	"UPI for electricity"
Beckn Protocol	Communication standard for energy apps	Like HTTP for websites
IES	India Energy Stack	Government's digital energy infrastructure
DISCOM	Distribution Company	The electricity company (BSES, Tata Power, etc.)
AMISP	Advanced Metering Infrastructure Service Provider	Company that manages smart meters
HES	Head-End System	The "brain" that collects all meter data
MDM	Meter Data Management	System that cleans and validates meter data
DLMS/COSEM	Meter communication protocol	The "language" smart meters speak
SHA-256	Hashing algorithm	Digital fingerprint generator
Hash chain	Linked hashes forming a tamper-evident log	A diary where each page references the previous
Blockchain	Distributed hash chain across multiple parties	A shared diary that nobody alone can change
Hyperledger Fabric	Permissioned blockchain framework	Private, invite-only blockchain network
Chaincode	Smart contract on Hyperledger	Automatic rules that execute when conditions are met
Escrow	Payment held by trusted third party	Like COD — payment released only after delivery
Net metering	Selling surplus to DISCOM only	Current system, low rates
DERC/UPERC	State electricity regulators	Government bodies that approve P2P trading
RWA	Resident Welfare Association	Society admin who drives adoption
DPDP Act	Data protection law	India's privacy law we must comply with

---

The simplest way to explain SolarShare to anyone:

"You know how UPI lets you send money to anyone? We're building the same thing, but for electricity. If your neighbor has solar panels and extra power, our app lets you buy it directly — cheaper for you, more profitable for them."